Recovered tools and trainers, part 2 of 2

Recovered tools and trainers, part 2 of 2

This part contains 20 notes, from `recovered-hacks/behaviour-models.md` through `recovered-hacks/TRAINER_RE_NOTES.md`.

`recovered-hacks/behaviour-models.md`

Trainer behaviour models

Date: 2026-11-01. Static read of the trainer images. Copies were unpacked with UPX where the packer was intact. No recovered PE was executed. No window was focused, and nothing was sent to a client.

These notes are the observable loops: which key, how many times, how long it waits, and whether it looks at a window title. `[bin]` is what was read out of the image or its bundled text. `[inf]` is a reading of that evidence, not a second measurement.

An earlier static pass already named AutoD, AutoIdle, AutoS, WaterGet v1.1, Random Walker, and the caps-lock utility. Those loops were read again here. They match. The new reads are Graal Load, the two AFK bots, Prayer and its helper, the window-retitle tool, Water Get v1.0, and the text sender. Digger is still truncated. The miner archive was not downloaded.

`reference_macros.py` is a table of the loops below. A unit test replays the table in this process. It does not open a socket, call a window API, or start a program.

What a server can see

Every sample below types into a classic client window, or writes text into one. None of them speaks the game protocol itself. A world checker sees the resulting actions: dig, use, walk, chat. The gaps below are the client-side waits, not a server cooldown.

AutoD `[bin]`

Yes/no box. No exits. A second box says it was made in 2001 and to stop it from the tray. Then, forever: sleep 1 millisecond, send `d`. No window check. No chat.

AutoS `[bin]`

Three lines. Repeat forever, send the two characters `s#`, end the repeat. The script sets no key delay and no sleep, and it does not look at a window.

AutoIdle `[bin]`

No sleep and no window check. One pass sends 48 arrow taps, then goes back to the start. The pass is left, right, six times; one left and three rights; left, left, left, right, right, right, four times; then left, right, five times. The exact list is `AUTOIDLE_KEYS` in the reference module.

Graal Auto Key `[bin]`

This is the program the other RpgWarrior tools call "the key". It does not send a game key.

A box says the operator has 5 seconds to focus the client. The script then sleeps 2 seconds (the box text and the sleep disagree). It reads the active window's title and renames that window to `Graal Auto`, then exits.

Tools that look for `Graal Auto` do nothing useful until this has run. WaterGet v1.1 does not look for that title.

WaterGet v1.1 `[bin]`

Ok/cancel box, author named in the box. Cancel exits. The script sets the key delay to 300 milliseconds and loops: send Down twice, send `a`, sleep 2000 milliseconds. It does not check a window title. The bundled howto is the operator's setup (water can, facing down on a source, client focused), not an extra key.

WaterGet v1.0 `[bin]`

Different loop from v1.1. It requires the window titled `Graal Auto`.

After the ready box it asks for two idle directions (the prompts' examples are Left and Right), tells the operator to equip a water can, and waits for `Graal Auto`. One pass, only while that window is active:

- key delay 100 milliseconds
- send `S`, sleep 1000 milliseconds, nineteen times
- sleep 2000 milliseconds
- key delay 50 milliseconds
- send the first direction twice, sleep 500 milliseconds
- send the second direction twice, sleep 500 milliseconds

If the window is not active it shows a waiting box, sleeps 1000 milliseconds, and resumes when the window is active. If the window was never renamed, it says to use the key first and exits.

Random Walker `[bin]`

Looks for a top-level window titled `Graal Auto`. A box says to focus it; the script sleeps 2000 milliseconds and exits with "use the key first" if that window is not active. An ok/cancel box uses the same wording as the water tool. Cancel exits.

The loop sets the key delay to 200 milliseconds and draws an integer from 0 through 5. Zero sends nothing. 1 is Up five times, 2 is Down five times, 3 is Left five times, 4 is Right five times, 5 is `S` five times. Between those checks, if the window is not active, it shows a waiting box and sleeps 1000 milliseconds.

Prayer `[bin]`

The script body was recovered. It is not still encrypted. The package is two macros.

`Graal Prayer` requires `Graal Auto`, same 2 second focus wait and "use the key first" exit as the walker. It asks for three operator strings. The prompts' examples are a prayer-weapon shortcut `R`, a naginata shortcut `E`, and an idle direction `LEFT`.

One pass, while `Graal Auto` is active:

1. Key delay 100. Send Tab.
2. Key delay 0. Three times: send the chat text `/use_skill praying` and Enter, then sleep 2000 milliseconds.
3. Sleep 500. Key delay 100. Send Tab. Sleep 500.
4. Send the naginata shortcut. Sleep 1000.
5. Start the helper in the `data` folder next to the script.
6. Key delay 0. Send the idle direction once.
7. Key delay 100. Send `s` ten times.
8. Sleep 500. Send the prayer-weapon shortcut. Sleep 500. Send Right. Sleep 500. Send the idle direction twice. Sleep 1000.

The same waiting box and 1000 millisecond sleep apply when the window is not active.

The helper (`data.exe`) sets the key delay to 100, sends Up ten times, and exits. It does not check a window. `[bin]`

Graal Load `[bin]`

VB6, title "Graal Load". The form label says to put the text files beside the program and to type `load<file.txt>` in the client. The example on the label is `load<bomb.txt>`.

The program enumerates windows and reads their text. For each line of the chosen file it writes that line into the target window, sends the Return key down (no key-up in that loop), and sleeps 150 milliseconds. It does not open a socket.

The bundled `bomb.txt` is 164 lines: appearance lines, a phrase typed out one fragment per line, a countdown from 100 down to 0, then `!!BOOM!!` and more appearance lines. That file is chat and costume text. The loader's timing does not depend on the words.

AFK Bot `[bin]`

VB6. The only imported runtime is the VB runtime, so the keystrokes go to whichever window the operator focused. The form timer interval is 3000 milliseconds. The readme says the operator has 3 seconds after Start to click the client chat box, or the bot turns itself off, and that it then types three phrases, one every 3 seconds, down the list.

The three default phrase boxes are `I am AFK`, `I am using an AFK Bot`, and `AFK Bot created by Zorg`. `[inf]` The readme's "down the list" is the order those boxes are filled, top to bottom as the operator sees them. The image stores the three strings; it does not store a sentence that says which box is first.

AFK Scroll Bot `[bin]`

Same author, same focus rule, different timer. The form timer interval is 100 milliseconds. A slider is labelled Wait Time. The readme says that slider is how many seconds the operator has to click the chat bar before typing starts, and that the bot then scrolls one phrase across chat. The default phrase box is `Scroll Text Here`.

The 100 millisecond timer is the scroll tick. The per-character shift is not a form string. The timer procedure moves the first character to the end; `scroll_phrase` in the reference module replays that, and

`remaining-2026-11-01.md` is the read. The start wait is the slider value times ten ticks. The image does not fix the slider.

Graal Text Sender `[bin]`

VB6, "Graal Text Sender" v1.3.5. The form timer interval is 100 milliseconds. The operator types the phrase and can set a timer per mode. No default phrase is stored.

Modes on the form: Word Scroller, Middle In-Out Letters, Word Stay, Two Words Repeat, Letter by Letter To-Alls, Letter by Letter Shout Outs. Word Stay has a message button and no timer button. The others have both. Each mode has On and Off.

The program looks up a window, and the client-name token is one of the strings it carries. It can send keys and send a window message. `GetAsyncKeyState` is declared. The key it would test is not in the resources, and nothing calls the check. `remaining-2026-11-01.md` is that read. There is no phrase list or log path beside it, so this is not a key logger.

The reference table does not include a GTS loop. The phrase and the per-mode wait are whatever the operator typed.

Not recovered

- **Digger.** The zip's compressed stream ends early. The surviving image is a partial UPX file (36111 bytes written, 41893 declared). The script block is not in the surviving bytes. The howto, which did survive, says to equip a shovel, stand on grass, focus the client, and click ok. It does not name the key or the wait. Confirmed again in `remaining-2026-11-01.md`. `[bin]`
- **Miner.** Not downloaded. The archive is the one the Drive abuse flag blocks.
- **Caps-lock utility.** The packed body was opened again. It stays packed. See `remaining-2026-11-01.md`.
- **Win32 listserver.** Already described. Not opened again.

`recovered-hacks/defense-checklist.md`

Operator defense checklist (Seihai / classic Graal worlds)

Companion to `DEFENSE_PROPOSALS.md` and `THREAT_TAXONOMY.md`. Tick before or right after going public. Items marked **[tools]** can grow into `seihai-servertools` checks/docs; **[gserver]** / **[listserver]** need runtime support.

Threat-intel confidence (second pass): **high** for Groups 1–3/5; **unknown** for Group 4 Miner (Drive-blocked); **medium** for stack-adjacent version faker / packet logger.

A. Before first public publish

- [ ] Run `seihai-servertools setup status` / go-public wizard; fix every **Block** finding.
- [ ] Confirm `localhost=false` and `serverinterface` is not loopback. **[tools]**
- [ ] Replace `YOURACCOUNT` in `staff` with the real owner account. **[tools]**
- [ ] Remove lab/test accounts from `accounts/`; rotate any password that ever appeared in docs or chat. **[tools]**

- [ ] Ensure listserver uses an **accounts file** (hashed), not open mode, when bound outside loopback. **[tools]**
- [ ] Firewall: client ports open; **gserver registration port** reachable only from your gservers until allow-list exists. **[listserver]**
- [ ] No UPnP unless you understand it; prefer manual port forward. **[tools]**
- [ ] Do **not** run classic third-party `ListServer.exe` tools (known password stealer in 2003 “Rick” build). **[T6]**
- [ ] Decide version policy: allow classic tokens, Seihai-only (`SEI00001`), or mixed. Document for players. **[T4]**
- [ ] Snapshot empty economy / account DB for later rollback.

B. Auth, sessions, staff

- [ ] Staff/RC/NC accounts: unique strong passwords; never a built-in default staff password.
- [ ] RC rights least-privilege; upload (`w`) only for LAT roles that need it.
- [ ] Prefer Seihai secure login for staff connections when available; pin server key fingerprint.
- [ ] Log and alert on RC/NC logins (account, IP, time). **[gserver]**
- [ ] Ban / lockout after repeated failed passwords. **[listserver]** / **[gserver]**

C. Anti-bot / anti-macro (Groups 1–3)

- [ ] Chat rate limit + auto-mute. **[gserver] [T1 / Group 1]**
- [ ] Online-time / AFK rewards require movement or real interactions — not chat alone. **[gserver]** / scripts
- [ ] Server-side cooldowns on water/dig/mine/pray/gather loops. **[gserver]** / NPC server **[T2 / Group 2]**
- [ ] Hourly/daily yield caps per account (and optionally per IP). **[gserver]**
- [ ] Nick / appearance change rate limits. **[gserver] [T3 / Group 3]**
- [ ] Review logs weekly for periodic chat (≈3s / letter-scroll) and single-tile interact storms.

D. Anti-cheat / anti-forge (T5)

- [ ] Movement: max speed, collision, level bounds enforced server-side. **[gserver]**
- [ ] Chests, items, combat damage authoritative on server. **[gserver]**
- [ ] Drop connections that send staff/RC/NC message ids on a normal player session. **[gserver]**
- [ ] Cap unknown message ids / oversized frames / messages per second. **[gserver]**
- [ ] Do not trust client version string alone; log token + behaviour mismatch. **[gserver]**
- [ ] World scripts: no secret rewards only the client can see; validate warps and gives. **[LAT]**

E. Content & LAT (T7)

- [ ] Run `seihai-servertools check WORLD` before publish and after big uploads. **[tools]**

- [ ] Review levels/NPCs whose names or scripts suggest staff escalation (`hacked*`, free items, unrestricted warps).
- [ ] LAT uploads: diff + conflict check; keep local backups of replaced files. **[tools]** / **[rc]**
- [ ] Production puts only from named staff accounts; retain put/delete logs.

F. Network & host

- [ ] Separate admin SSH/RDP from game ports; key-only SSH where possible.
- [ ] TLS termination is N/A for classic Graal TCP — compensate with secure login + firewall.
- [ ] Backups of `accounts`, levels, and NPC DB on a schedule; test restore once.
- [ ] Host clock synced (log correlation).

G. Detection quick tests (lab or quiet hours)

- [ ] From a test account, flood chat → expect mute/kick (once limits exist).
- [ ] Sustained interact on one tile → expect cooldown/cap.
- [ ] Impossible speed / out-of-level position → expect reject (once authority exists).
- [ ] Login with `localhost` misconfig on a staging bind → wizard/check must **Block**.

H. Incident response (short)

1. Snapshot logs + account row + recent inventory.
2. Kick/mute/ban; throttle IP if multi-account.
3. Freeze economy actions if item dupe suspected; diff vs last backup.
4. Rotate staff and listserver secrets if credential theft possible.
5. File a short postmortem in `docs/lab/` or world ops notes (no passwords).

I. Patch priorities (owners → stack maintainers)

Priority	Ask
P0	Keep using `seihai-servertools` safety checks; never publish with Blocks
P0	Firewall listserver gserver port; hashed accounts
P1	Chat + gather rate limits / cooldowns on gserver (Groups 1–2)
P1	Server-authoritative movement and items (T5)
P2	Version/behaviour mismatch kicks; secure login option
P2	`lat-core` script lints + richer `check` (T7)
P3	Automated log queries / “lights” for abuse signals in the GUI

Static analysis only. Do not execute recovered trainers or patched clients against public servers. Do not commit hack binaries into seihai-servertools.

`recovered-hacks/DEFENSE_PROPOSALS.md`

Defense proposals — recovered Graal hacks / trainers

Date: 2026-10-31 (third pass; inventory + taxonomy + Ghidra RE)
Audience: operators of Seihai-stack worlds (`seihai-gserver` / `seihai-gserver-v2`, `seihai-listserver`, staff RC/NC) and maintainers of `seihai-servertools`.
Method: static analysis only (Drive listing, archive inventory, readmes, PE/UPX/strings, Ghidra headless). No hacks executed; no exploit PoCs.
Companions: `THREAT_TAXONOMY.md` (Groups 1–5 + RE addendum) · `defense-checklist.md` · `INVENTORY.md` · `TRAINER_RE_NOTES.md` · `ghidra-analysis/`

Intended long-term home: mirror under `seihai-servertools/docs/security/` (local draft prepared; that repo returned 403 to this environment's push token).

Confidence and gaps

Item	Level	Notes
Overall threat picture for Drive `Hacks/Trainers`	**High**	`INVENTORY.md` + `THREAT_TAXONOMY.md`: 18/20 archives, 24 unique PEs, readmes + VB6/AutoIt strings agree
Taxonomy Groups 1–3, 5	**High**	AFK bots, AutoIt macros, chat/appearance spam, capslock
Group 4 (`Graal_Miner.zip`)	**Unknown payload**	Drive-blocked; treat hostile until RE — **skipped this pass**
Prayer / `data.exe` / GTS / AFK Bot (static RE)	**High**	`TRAINER_RE_NOTES.md` + `ghidra-analysis/*/SUMMARY.md`
Digger (truncated UPX)	**Medium**	Hash + AutoIt v2.63 cues; imports/script not recoverable
Version faker / CRC / packet logger (adjacent archives)	**Medium**	Not in this Hacks tree
Classic login / RC / stealer ListServer	**High**	Seihai docs + `Graal_analysis`
Network injectors / memory editors beyond these archives	**Unknown**	Not in Hacks listing

Sources used

- Owner private Drive folder `Hacks → Trainers/` (+ `AutoPrograms by RpgWarrior/`) — re-listed 2026-10-31 (Drive URL omitted)
- `INVENTORY.md` / local static extracts (UPX AutoIt `SEND`/`keybd_event`; VB6 AFK/GTS/GraalLoad strings)
- `THREAT_TAXONOMY.md` Groups 1–5 + RE addendum
- `TRAINER_RE_NOTES.md` — Ghidra/static IOC synthesis
- Adjacent client archives: v4 packet-logger, v5 CRC / "Version 5.120 Faker Hack"
- `seihai-servertools` go-public / `setup-core` checks; `AGENTS.md` stealer ListServer warning

Threat summary (operator view)

1. **Group 1 — AFK chat bots** — VB6 timers into Graal chat (Zorg AFK / Scroll). Normal chat packets, unnatural periodicity (~3s).
2. **Group 2 — RpgWarrior / AutoIt gameplay macros** — Water/Dig/Pray/Walk/Key + autoprogs; AutoIt `SEND` loops.

3. **Group 3 — Chat / appearance script spam** — GTS scrollers; GraalLoad + `bomb.txt` via `SendMessageA`.
4. **Group 4 — Suspected malware (Miner)** — Drive `cannotDownloadAbusiveFile`; host risk unknown until obtained.
5. **Group 5 — Benign / misfiled** — `capslock.exe` (ASPack CapsLock toggler).
6. **Stack-adjacent (outside Hacks/)** — version faker / packet logger; cleartext classic passwords; stealer ListServer; `localhost=true`; open gserver registration; hostile world content.

Technique cards

T1 — AFK / chat bots (Group 1; also GTS in Group 3, autoprogs idle in Group 2)

Evidence
AFK Bot readme + RE strings: default phrases `I am AFK` / `I am using an AFK Bot`, UI `AFK Bot created by Zorg`, warning about illegality on playerworlds; ~3s timer after focusing chat. Scroll Bot: growing/scrolling string. GTS (see T3): Word Scroller / letter-by-letter shouts (`MSVBVM60.DLL`). Autoprogs share UPX/AutoIt markers with Group 2. **No Winsock** on AFK Bot static surface — client-side chat abuse only.

How it works (high level)
External Windows programs inject keystrokes into the focused Graal chat field on a timer to farm online-time / stay “active”.

Detection — fixed inter-arrival chat; phrase/scroll patterns (`I am AFK` cluster); online-time with near-zero movement/combat. Host: `AFK Bot.exe` + Zorg UI strings.
Prevention — messages/min caps; mute→kick; AFK rewards require movement/interaction, never chat alone; NPC scripts must not trust client timers.
Response — mute/warn/kick; snapshot chat timestamps; rebalance farmed NPC rewards.
seihai-servertools — policy docs here; optional offline periodic-chat log parsers later; enforcement in gserver.

T2 — Activity automation (Group 2; Group 4 Miner blocked)

Evidence
Howtos: equip → stand on resource → start EXE → focus Graal. **RE (Prayer / `data.exe`):** twin AutoIt v2.64 UPX macros — APIs `keybd_event`, `mouse_event`, `SendMessageA`, `PostMessageA`, `FindWindowA`, `OpenProcess`, `VirtualAlloc`; **no** `WSAStartup` / WinINet / Winsock and **no** classic injection (`WriteProcessMemory` / `CreateRemoteThread`). Encrypted AutoIt overlays differ (~924 B vs ~180 B) — keep both packed + unpacked hashes. **Digger:** truncated zip PE (36111 / 41893 bytes); UPX overlay corrupt (`upx` refuses unpack); Ghidra import table empty; still classifiable as AutoIt v2.63 + UPX via strings + howto — hash `876723732378b36e…` usable, behaviour not recoverable until complete PE. Miner ×2 Drive-blocked (~39156 bytes) — **skipped**; taxonomy keeps Group 4 host-risk open.

Detection — sustained interact/dig rates; low movement entropy at resource tiles; single-action-type sessions; shared IP timing fingerprints. Host: UPX, the compiled AutoIt 2.6 banner, the AutoIt script-start marker, and `keybd_event` (no network imports).
Prevention — server-authoritative gather/prayer/dig/water cooldowns; yield caps per account/IP; soft anti-macro (facing/inventory/movement between gathers).
Response — soft yield nerf; hard kick/temp ban; wipe ill-gotten stacks if needed.
seihai-servertools — `lat-core/check.rs` lint for unbounded resource scripts; go-public warn on bot-farm lab leftovers.

T3 — Chat / appearance spam (Group 3: GTS + GraalLoad + `bomb.txt`)

Evidence
GTS RE: VB6 DarkShadowz **Graal Text Sender v1.3.5**; late-bound `FindWindowA`/`Ex`, `OpenProcess`, `GetAsyncKeyState`, `SendMessageA`, module `SendKeys1`; timer UI (Word Scroller / shout / to-all); **no Winsock**. Host combo IOC: VB6 + those APIs + product strings and the author's homepage (host omitted). GraalLoad: a load command that names a script file, plus `SendMessageA`. `bomb.txt`: setnick/setbody + countdown chat (not an explosive).

Detection — sub-human nick/appearance/chat bursts; prop churn without movement; GTS flood modes (scroller / to-all / shout). Host: window title `Graal Text Sender`.
Prevention — nick/appearance rate limits; T1 chat flood controls; rate-limit shout/to-all.
Response — mute; revert nick/appearance; escalate harassment.

T4 — Version fakery + CRC patch (stack-adjacent)

Evidence — named v5 CRC / “Version 5.120 Faker” archives outside this Hacks folder.

Detection — version token vs behaviour mismatch; prefer `SEI00001` secure login on Seihai-only worlds.
Prevention — **do not trust version string alone**; behaviour allowlists; optional capability challenge.
Response — generic “outdated or modified client” disconnect; ban repeats.

T5 — Packet logger / forged packets (stack-adjacent)

Evidence — v4 “+Packet Logger” client archives.

Prevention (highest priority for a real gserver)

Control	Where
Server-authoritative movement / inventory / chests / combat	gserver (+ NPC server)
Reject RC/NC-only message ids on player sessions	gserver
Unknown-id budget, max frame size, max messages/sec	gserver
No built-in default staff password; separate RC/NC auth	gserver + servertools checks
Secure login `SEI00001`	gserver + listserver
No secrets in client-visible scripts	LAT / world content

seihai-servertools — extend public-profile checks (lab accounts, default staff passwords; `localhost=true` Block already exists); LAT denylist uploads; **never run classic ListServer.exe**.

T6 — Credential theft & unsafe hosting (stack-adjacent)

Cleartext classic login; 2003 “Rick” ListServer stealer (`AGENTS.md`); `localhost=true` public bind; open gserver registration; lab accounts left public.

Prevention — hashed listserver accounts; firewall gserver registration port; pin `SEI00001` key; wipe lab accounts on publish.
Response — rotate secrets; if stealer ListServer ran on admin PC, burn all saved Graal client passwords.

T7 — Hostile world content (LAT)

Levels/NPCs that escalate rights when scripts are trusted. Filename ≠ proof.

Prevention — `seihai-servertools check`; GS1 lints; dual-control RC `w` uploads; put/delete logs.

Priority patch / build order

1. Keep/extend go-public **Block**s (`setup-core/checks.rs`).
2. Gserver chat + gather rate limits / cooldowns (defeats Groups 1–2 / T1–T3).
3. Gserver movement / inventory / chest authority (defeats T5).
4. Listserver gserver allow-list + hashed accounts.
5. Offer `SEI00001` secure login; document tradeoff.
6. LAT world linter in `lat-core/check.rs`.
7. Ops runbooks — this doc set; mirror to `seihai-servertools/docs/security/` when writable.
8. Log queries for periodic chat, gather loops, version mismatches.

Concrete controls mapped to `seihai-servertools`

Defense	Where
Unsafe public bind / localhost / UPnP	`setup-core` checks (**exists**)
Host-listserver open accounts on outside bind	listserver checks (**exists**)
Lab leftovers / default accounts	`check_server_folder`
Operator hardening + checklist	this folder (and `docs/security/` in servertools)
World script / link / gmap safety	`lat-core/check.rs`
Packet/rate policy	**gserver** (not servertools)

Do not commit trainer binaries, packet loggers, or CRC patches into MIT crates. Keep under `recovered-hacks/` (gitignored except `*.md`).

Suggested monitoring signals

Signal	Suggest alert
Chat msgs / account / min	> 20 sustained 5+ min
Identical message repeats	≥ 10 in 5 min
Interact/use / min at fixed tile	> 40 for 10+ min
Speed > configured max	any
Unknown message ids	> 3 / session
Version token ∉ allowlist	info; kick if strict
RC login from non-staff IP	page owner
New gserver registration	page listserver host
Failed passwords from many IPs	lockout

Key host / string IOCs (from RE)

Family	IOC cluster
Prayer / `data.exe`	UPX + AutoIt v2.64; `keybd_event` / `mouse_event` / `SendMessageA`; **no Winsock**; dual packed+unpacked SHA-256 in `TRAINER_RE_NOTES.md`
Digger	Truncated UPX PE; AutoIt v2.63 splash fragment; hash only until complete sample
GTS	VB6 + `FindWindow*` + `OpenProcess` + `SendMessageA` + `GetAsyncKeyState`; title `Graal Text Sender`
AFK Bot	Strings `I am AFK` / `I am using an AFK Bot` / Zorg UI; VB6 + `Timer1`

Full tables: `TRAINER_RE_NOTES.md`. Per-binary: `ghidra-analysis/*/SUMMARY.md`.

Residual gaps (Ghidra / RE siblings)

Priority	Gap
P0	Obtain `Graal_Miner.zip` (`acknowledgeAbuse` / mirror) + malware triage — **still skipped**
P1	Prayer / `data.exe` AutoIt **script body** recovery (overlays still encrypted)
P1	Digger truncated UPX — clean mirror or complete PE
P2	~~GTS `OpenProcess` / `GetAsyncKeyState`~~ — **done:** SendMessage/ hotkey automation, not proven inject
P2	autoprogs script recovery — macros only?
P3	Remaining RpgWarrior SEND loops; GraalLoad confirm
Adjacent	Version-faker / packet-logger disassembly

Inventory + taxonomy + RE notes are enough to ship operator defenses **now**; remaining RE refines AutoIt script plaintext and Group 4 host-risk, not the patch order above.

`recovered-hacks/ghidra-analysis/AFK_Bot/SUMMARY.md`

Defense RE — `AFK Bot.exe` (representative AFK chat bot)

Static analysis only — binary was never executed.
Analyzed (UTC): 2026-10-31
Ghidra: 12.2 headless
Input: `/workspace/recovered-hacks/Trainers/extracted/AFKBOT/AFKBOT/AFK Bot.exe`
Sibling (not fully re-run): `AFK Scroll Bot.exe` — same Zorg/VB6 family

Identity

Field	Value
SHA-256	`35dbd87ca7560712aae0e1bb3c4b4861e5233eac32299170af25eaa71c13ff35`
Size	24576
PE TimeDateStamp	`1032629018` (2002-09-21T17:23:38Z)
Archive stamp	~2014 on extracted tree
Format	PE32 GUI, 3 sections
Runtime	Visual Basic 6 (`MSVBVM60.DLL` only in PE import dir)
Author cue	**Zorg** — “AFK Bot created by Zorg”

Imports / APIs

Ghidra lists **only** `MSVBVM60.DLL`. No explicit `user32`/`kernel32` names in the import table (unlike GTS). Chat send path is almost certainly VB intrinsic **SendKeys** / late-bound calls not listed as named imports — weaker host API IOC than GTS.

UI/timer: form control `Timer1`, buttons Start, status `Sent 0 AFK Messages`.

String IOCs (high value)

String
`AFK Bot`
`AFK Bot created by Zorg`
`I am using an AFK Bot`
`I am AFK`
`WARNING : AFK Bots are illegal on some playerworlds`
`AFK = Away from Keyboard`
`Not currently AFK`
`Sent 0 AFK Messages`
Build path: `C:\Program Files\Microsoft Visual Studio\VB98\Source\AFK Bot\Project1.vbp`

Behavioral tells

- Operator focuses Graal chat; bot timer emits rotating AFK phrases (readme-documented pattern in taxonomy Group 1).
- Presence / online-time farming + chat noise; **no network imports** observed.
- Server-side: identical periodic chat lines matching default phrases above.

Ghidra notes

Metric	Value
Functions	6 (5 internal)
Imports	45 (MSVBVM60 only)
Interesting strings	8

Small VB GUI; decomp confirms thin wrapper around VB runtime — detection should key off **resources/strings**, not exotic imports.

Artifacts

- `imports.txt`, `strings_interesting.txt`, `decompiled/`, `prelim/`, `logs/`
- Strings also: `../../_static_extract/autoit_scripts/AFK_Bot.exe.allstrings.txt`

`recovered-hacks/ghidra-analysis/Graal_Digger/SUMMARY.md`

Defense RE — `Graal Digger.exe`

Static analysis only — binary was never executed.
Analyzed (UTC): 2026-10-31
Ghidra: 12.2 headless (limited — corrupt UPX)
Input: `/workspace/recovered-hacks/Trainers/extracted/Graal_Digger/Graal Digger.exe`
Howto: `/workspace/recovered-hacks/Trainers/extracted/Graal_Digger/Graal_Digger-Howto.txt` (author **Selzar**)

Identity

Field	Value
SHA-256 (carved PE)	`876723732378b36e8fd31d09120ec5a1b584cf46fb672ab9d5cf7a0a4f8b56d4`
Size on disk	36111 (truncated; zip declares uncompressed **41893**)
PE TimeDateStamp	`1012939264` (2002-02-05T20:01:04Z)
Format	PE32 GUI, **UPX compressed**, 3 sections (`UPX0`/`UPX1`/`.rsrc`)
Runtime cue (packed strings)	`AutoIt` + mangled `_SCRIPTFULLPATH`… / splash fragment mentioning **v2.63**

Zip / packer blockers

1. **`Graal_Digger.zip` dual local entries:** first local header is `Graal_Digger-Howto.txt` (219 B); second is `Graal Digger.exe`. Central directory naming historically mismatched howto vs exe (taxonomy “zip quirk”).
2. **Deflate stream truncated:** recovered PE is 36111 B vs declared 41893 B (~5782 B missing) → CRC incomplete.
3. **`upx -d` / `upx -t`:** `OverlayException: invalid overlay size; file is possibly corrupt`.
4. **Ghidra:** imports **0**, interesting strings **0**, only a UPX stub entry recovered — expected for unbroken packed+truncated image.

Static signals still usable for defense

Signal	Detail
Packer	`UPX!` at file offset 992; section names `UPX0`/`UPX1`
Family	AutoIt (v2.63 splash fragment in compressed stream)
Howto behavior	Equip shovel → grass → start tool → focus Graal → OK to start dig loop
Filename IOC	`Graal Digger.exe`, `Graal_Digger-Howto.txt`
Expectation if intact	Same class as other RpgWarrior macros: `keybd_event` / dig key loop after focus — **not confirmed** on this truncated image

`objdump` cannot map the UPX import directory (`.rsrc` claimed larger than available bytes). No Winsock strings observed in the truncated blob.

Defense guidance

- Treat sample as **corrupt/incomplete AutoIt+UPX trainer**, not a confirmed second-stage malware drop — but do **not** execute; incompleteness can also indicate intentional anti-analysis packaging.
- Host detections: hash above + UPX+AutoIt combination + howto filename co-occurrence.
- Server-side: repetitive dig / shovel-use timing from focused classic client (same as Water Get / AutoD family).

Artifacts

- Ghidra stub export under this directory (`SUMMARY` previously salvage-oriented; this file is the defense summary).
- `prelim/strings_all.txt`, `_analyze.log`
- Related: howto text; zip under `Trainers/Graal_Digger.zip`

`recovered-hacks/ghidra-analysis/Graal_Prayer/SUMMARY.md`

Defense RE — `Graal Prayer.exe`

Static analysis only — binary was never executed.
Analyzed (UTC): 2026-10-31
Ghidra: 12.2 headless via `/workspace/tools/ghidra/analyze_tool.sh`
Input analyzed: UPX-unpacked copy `/workspace/recovered-hacks/_static_extract/upx_unpacked/Graal_Prayer_Graal_Prayer.exe`
Packed source: `/workspace/recovered-hacks/Trainers/extracted/Graal_Prayer/Graal Prayer.exe`

Identity

Field	Value
Packed SHA-256	`6959bcb8066efdee0dd842b6ee9d769cf93230d90a4ebca04786bfddb7f88860`
Unpacked SHA-256	`a348ecafc3fdf3069dcaf9dd6d776c3f83c6f6d4f547ccf09d94f3694d952074`
Packed size	43420
Unpacked size	86940
PE TimeDateStamp	`1040239050` (2002-12-18T19:17:30Z) — shared with sibling RpgWarrior builds
Format	PE32 GUI, Intel 80386
Packer / runtime	UPX, then the AutoIt 2.64 compiled-script runtime

Package role

Ships with sibling `data/data.exe` in the same zip (`Graal_Prayer.zip`). Both are independent AutoIt v2.64 EXEs (identical import sets; identical PE timestamp). No plaintext `data.exe` / path string was found in either image — dual layout is packaging, not an obvious PE dropper string. Encrypted script overlays differ in length (this EXE ~924 B vs `data.exe` ~180 B), so they are **two scripts**, not byte-identical copies.

Defense-relevant capabilities (imports / strings)

Input / window automation (USER32): `keybd_event`, `mouse_event`, `SendMessageA`, `PostMessageA`, `SendMessageTimeoutA`, `FindWindowA`, `GetForegroundWindow`, `SetForegroundWindow`, `EnumWindows`, `GetWindowTextA`, `SetTimer` / `KillTimer`, `BlockInput`, `GetKeyState`, `VkKeyScanA`.

Process / memory (KERNEL32): `OpenProcess`, `CreateProcessA`, `VirtualAlloc` / `VirtualProtect` / `VirtualFree`, `TerminateProcess`, `Sleep`.
Not imported: `WriteProcessMemory`, `CreateRemoteThread`, `SetWindowsHook*`, `SendInput`, any Winsock / WinINet / WinHTTP API.

Other: `RegOpenKeyExA` family (ADVAPI32); `Shell_NotifyIconA`; `ExitWindowsEx` (AutoIt runtime capability — shutdown). Profile APIs `Get/WritePrivateProfileStringA` present (INI-style config capability).

AutoIt markers: the compiled-script banner for AutoIt 2.64, the script-start marker, a powered-by line that points at the AutoIt vendor site (host omitted), the `SETKEYDELAY` and `HIDEAUTOITWIN` directives, and the `A_SCRIPTDIR` / `A_SCRIPTNAME` variables.

Behavioral tells (for detection)

- UPX section names `UPX0`/`UPX1` on disk sample; after unpack, classic AutoIt v2 string cluster.
- Likely focuses Graal client then synthesizes keyboard/mouse via `keybd_event` / `mouse_event` (not raw `SendInput`).
- Timer-driven loop (`SetTimer`) typical of AFK prayer/emote macros.
- No network import surface — host C2 unlikely from this PE alone; risk is **game automation**.

Script recovery status

Embedded AutoIt script body is **encrypted** in the PE overlay (high-entropy blob after `.rsrc`, shared 16-byte family header `a3484bbe…` across RpgWarrior unpacks). Plaintext script lines were **not** recovered; only runtime keyword tables after the script-start marker (e.g. `RWINUP`, key names). Treat payload as "AutoIt input macro" from packaging/name + API set.

Ghidra notes

Metric	Value
Functions	303 (301 internal)
Imports	158
Interesting strings (export filter)	34

Decompilation is mostly CRT / AutoIt interpreter glue, not a readable user script. Prefer strings + imports for defense IOCs over deep decomp.

Artifacts

- `imports.txt`, `exports.txt`, `functions.txt`, `strings_interesting.txt`
- `decompiled/`, `prelim/`, `logs/`, `summary_stats.txt`
- Sibling notes: `../Graal_Prayer_data/SUMMARY.md`

`recovered-hacks/ghidra-analysis/Graal_Prayer_data/SUMMARY.md`

Defense RE — `Graal Prayer` / `data/data.exe`

Static analysis only — binary was never executed.
Analyzed (UTC): 2026-10-31
Ghidra: 12.2 headless
Input analyzed: UPX-unpacked copy
`/workspace/recovered-hacks/_static_extract/upx_unpacked/Graal_Prayer_data_data.exe`
Packed source: `/workspace/recovered-hacks/Trainers/extracted/Graal_Prayer/data/data.exe`

Identity

Field	Value
Packed SHA-256	`c331eee0e985c903db933432a62a4972b835aefe9a4832d9ce5705c694a86145`
Unpacked SHA-256	`b72e36502b2638ceeb9642ecb8d9da3c08c880175e60ec782a48b9ae93172a4c`
Packed size	42676

Field	Value
Unpacked size	86196
PE TimeDateStamp	`1040239050` (same as `Graal Prayer.exe`)
Format	PE32 GUI, Intel 80386
Packer / runtime	UPX, then the AutoIt 2.64 compiled-script runtime

Relationship to wrapper

- Same AutoIt runtime / identical import table as `Graal Prayer.exe`.
- PE image identical for the first **86107** bytes of the unpacked files; divergence is in the trailing encrypted overlay / resources (`overlay` ≈ **180 B** here vs ≈ **924 B** on the wrapper).
- No cross-references to `Graal Prayer.exe` or `data\` paths in strings.
- Defense interpretation: second compiled macro in the same distribution (helper/alternate script), **not** a confirmed loader for a third-stage PE. Still elevates dual-EXE packaging as an IOC worth hashing both files.

Defense-relevant capabilities

Same as wrapper (AutoIt v2.64 stock imports):

- Input: `keybd_event`, `mouse_event`, `SendMessageA`, `PostMessageA`, `FindWindowA`, `SetTimer`/`KillTimer`, `BlockInput`
- Process: `OpenProcess`, `CreateProcessA`, `VirtualAlloc` (no `WriteProcessMemory` / `CreateRemoteThread`)
- Network APIs: **none** observed
- Markers: the compiled-script banner for AutoIt 2.64, the AutoIt script-start marker, and the AutoIt vendor site (host omitted)

Behavioral tells

- Filename `data.exe` under `data/` is itself a weak host IOC (paired with `Graal Prayer.exe`).
- Smaller encrypted overlay ⇒ shorter/simpler script than the wrapper (cannot confirm actions without decrypt).
- Same PE timestamp cluster as other Mar-2003 RpgWarrior UPX AutoIt trainers.

Ghidra notes

Metric	Value
Functions	303 (301 internal)
Imports	158
Interesting strings	34

Same interpreter-shaped call graph as the wrapper; no unique plaintext Graal window title recovered.

Artifacts

- `imports.txt`, `exports.txt`, `functions.txt`, `strings_interesting.txt`
- `decompiled/`, `prelim/`, `logs/`, `summary_stats.txt`
- Sibling: `../Graal_Prayer/SUMMARY.md`

recovered-hacks/ghidra-analysis/GTS/SUMMARY.md

Defense RE — `GTS.exe` (Graal Text Sender)

Static analysis only — binary was never executed.
Analyzed (UTC): 2026-10-31
Ghidra: 12.2 headless
Input: `/workspace/recovered-hacks/Trainers/extracted/GTS/GTS.exe`

Identity

Field	Value
SHA-256	`36e264ff1b9bd441b278ec4afc265c7991dc2f9ae065f721d178f59c74d2d0ab`
Size	40960
PE TimeDateStamp	`1043589876` (2003-01-26T14:04:36Z)
Format	PE32 GUI, 3 sections (`.text`/`.data`/`.rsrc`) — **not UPX**
Runtime	Visual Basic 6 (`MSVBVM60.DLL`)
Title / author	**Graal Text Sender by DarkShadowz v1.3.5**

Imports (defense view)

PE import directory (Ghidra): only `MSVBVM60.DLL` (VB runtime + ordinals).
Late-bound / string-visible Win32 APIs (via VB `DllFunctionCall` / declare):

API	Likely use
`FindWindowA` / `FindWindowExA`	Locate Graal (or chat) HWND
`GetWindowThreadProcessId`	Resolve PID for window
`OpenProcess`	Open target process handle
`GetAsyncKeyState`	Hotkey / key-state polling
`SendMessageA`	Inject chat / UI messages
`Shell_NotifyIconA`	Tray icon
`ShellExecuteA`	Launch URL/help
`BitBlt` / `TransparentBlt`	UI blitting
`GetTickCount`	Timing
`CloseHandle`	Handle cleanup

Module name string: `SendKeys1` — VB-style SendKeys helper. UI frames: Word Scroller, Two Words Repeat, Letter-by-Letter shout/to-all, timer controls (`Set Timer`, `Timer1`).

Not observed: Winsock / WinINet imports; AutoIt markers; UPX sections.

Window / branding IOCs

String	Notes
`Graal Text Sender`	Window / product title
`Graal Text Sender by DarkShadowz v1.3.5`	Version banner
`Graal`	Standalone title token (FindWindow target candidate)
`This lil Graal Proggy was made by DarkShadowz`	About text
the author's homepage (host omitted)	Author URL

String	Notes
`C:\Program Files\Microsoft Visual Studio\VB98\Graal\Project1.vbp`	Build path leftover
`(To-All Messages Sent)`	Feature string

Behavioral tells

- Timer-driven chat spam / scroller modes into focused or found Graal window.
- Combination **`OpenProcess`** + **`GetAsyncKeyState`** + **`FindWindow*`** + **`SendMessageA`** is the strongest host signature for this tool (VB6 + those API names).
- Social harm: shout / to-all / letter spam — playerworld chat abuse; not host ransomware-class from static view.

Ghidra notes

Metric	Value
Functions	34 (30 internal)
Imports listed	76 (all MSVBVM60)
Interesting strings	22

Decompiled VB thunks are low-signal; prefer string/API IOCs.

Artifacts

- `imports.txt`, `strings_interesting.txt`, `decompiled/`, `prelim/`, `logs/`
- Full string dump also at `../../_static_extract/autoit_scripts/GTS.exe.allstrings.txt`

`recovered-hacks/INVENTORY.md`

Recovered Hacks Inventory

Source: Owner private Google Drive folder named `Hacks` (URL and folder id omitted from this repo). **Listed at:** 2026-10-31T20:49:06.899885+00:00 **Handling:** Contents treated as untrusted. Download agent did not execute binaries/scripts; static inspection only. Sibling agents may have added _static_extract/ artifacts.

Priority analysis targets

Deep-analysis queue for sibling agents (static only; do not execute unknowns):

1. **Trainers/Graal_Miner.zip (+ AutoPrograms duplicate)** *blocked_download*
   - Google Drive hard-blocks with cannotDownloadAbusiveFile. Obtain via acknowledgeAbuse=true or owner “Download anyway” for malware triage.
2. **Trainers/extracted/capslock/capslock.exe** `2576cbeacd5ed30f…`
   - Largest unique PE (10 sections, non-UPX). Custom loader stub (VirtualAlloc/GetProcAddress). Check packing and any input-capture beyond CapsLock macros.
3. **Trainers/extracted/Graal_Prayer/data/data.exe** `c331eee0e985c903…`
   - UPX-packed sidecar; AutoIt compiled-script strings. Secondary helper — see _static_extract/ upx_unpacked and autoit_scripts.

4. **Trainers/extracted/GraalLoad/Graal Load.exe + bomb.txt** `812a0d32da379e75…`
    - VB6 window-message automation (FindWindow/SendMessage). bomb.txt is a Graal appearance/chat theatrical script, not an explosive payload.
5. **Trainers/extracted/GTS/GTS.exe** `36e264ff1b9bd441…`
    - VB6 “Graal Text Sender” chat automation (scrollers/shouts).
6. **Trainers/extracted/autoprogs/{AutoIdle,AutoD,AutoS}.exe**
    - UPX 1.07 family with AutoIt indicators. Sibling unpacks already under _static_extract/upx_unpacked/.
7. **RpgWarrior UPX trainers (Auto Key, Digger, Prayer, Random Walker, Water Get)**
    - Classic Graal automation. Digger PE truncated from corrupt zip. AutoPrograms copies are byte-identical archives.
8. **AFKBOT / AFKSCROLLBOT (Zorg)**
    - Unpacked PE32 chat-AFK bots with author readmes. Confirm SendKeys-style vs injection.

Summary

- Drive files listed: **20**
- Downloaded OK: **18**
- Blocked by Google: **2**
- Archives extracted: **18** (originals preserved)
- Extracted member files: **30**
- Unique primary PE binaries: **15** (22 extracted instances incl. duplicates)
- Sibling `_static_extract/` artifacts: **53**

On-disk vs git

Binaries and archives remain on the agent workspace under `recovered-hacks/Trainers/` for sibling static analysis. Repo `.gitignore` tracks analysis docs/metadata only (`*.md`, `*.json`, `*.BLOCKED.txt`), not PE/archives.

Blockers

- `Trainers/Graal_Miner.zip` — **403 cannotDownloadAbusiveFile — Google identified as malware or spam** (Drive file id omitted; expected 39156 bytes)
- `Trainers/AutoPrograms by RpgWarrior/Graal_Miner.zip` — **403 cannotDownloadAbusiveFile — Google identified as malware or spam** (Drive file id omitted; expected 39156 bytes)
    - Same reported size as Trainers/Graal_Miner.zip; likely identical duplicate

Owner workaround: Drive API download with `acknowledgeAbuse=true`, or Drive UI “Download anyway”.

Key paths

```
recovered-hacks/
├── INVENTORY.md / inventory.json
├── DEFENSE_PROPOSALS.md / defense-checklist.md   # sibling notes
├── _static_extract/                              # sibling UPX/strings/AutoIt extracts
└── Trainers/
    ├── *.zip *.7z                                # Drive originals (18)
    ├── Graal_Miner.zip.BLOCKED.txt
    ├── extracted/                                # unpacked siblings
```

```
    └── AutoPrograms by RpgWarrior/
        ├── *.zip + Graal_Miner.zip.BLOCKED.txt
        └── extracted/
```

Observations

- Collection is vintage Graal Online trainers/bots (timestamps ~2002–2017), not general exploit kits.
- Many PEs are UPX-compressed; capslock.exe is a multi-section custom loader.
- GraalLoad (VB6) loads client script .txt files; bomb.txt is theatrical chat/appearance scripting.
- GTS = Graal Text Sender (VB6 chat macros).
- Prayer data/data.exe shows AutoIt compiled-script strings; sibling unpacks present.
- Graal_Digger.zip is damaged; recovered PE is truncated (CRC fail).
- AutoPrograms by RpgWarrior mirrors Trainers archives (byte-identical where both present).
- Sibling analysis artifacts live under _static_extract/ (UPX unpacks, strings, AutoIt script extracts) plus DEFENSE_PROPOSALS.md / defense-checklist.md.

Duplicate archives (analyze once)

- `Trainers/Graal_Auto_Key.zip` ≡ `Trainers/AutoPrograms by RpgWarrior/Graal_Auto_Key.zip`
- `Trainers/Graal_Digger.zip` ≡ `Trainers/AutoPrograms by RpgWarrior/Graal_Digger.zip`
- `Trainers/Graal_Prayer.zip` ≡ `Trainers/AutoPrograms by RpgWarrior/Graal_Prayer.zip`
- `Trainers/Graal_Random_Walker.zip` ≡ `Trainers/AutoPrograms by RpgWarrior/Graal_Random_Walker.zip`
- `Trainers/Graal_Water_Get.zip` ≡ `Trainers/AutoPrograms by RpgWarrior/Graal_Water_Get_v1.0.zip`
- `Trainers/Graal_WaterGet.zip` ≡ `Trainers/AutoPrograms by RpgWarrior/Graal_Water_Get_v1.1.zip`

Unique primary PE binaries

SHA256 (16)	Size	file(1)	Paths
463e8f757e82bc5a…	42793	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Auto_Key/Graal Auto Key.exe Trainers/extracted/Graal_Auto_Key/Graal Auto Key.exe
876723732378b36e…	36111	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Digger/Graal Digger.exe Trainers/extracted/Graal_Digger/Graal Digger.exe
6959bcb8066efdee…	43420	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Prayer/Graal Prayer.exe Trainers/extracted/Graal_Prayer/Graal Prayer.exe

SHA256 (16)	Size	file(1)	Paths
c331eee0e985c903…	42676	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Prayer/data/data.exe Trainers/extracted/Graal_Prayer/data/data.exe
b41973d19fc0089c…	43185	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Random_Walker/Graal Random Walker.exe Trainers/extracted/Graal_Random_Walker/Graal Random Walker.exe
4f95fa2af7d7db1c…	43483	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Water_Get_v1.0/Graal Water Get.exe Trainers/extracted/Graal_Water_Get/Graal Water Get.exe
7482de4824d370f1…	41820	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Water_Get_v1.1/Graal WaterGet.exe Trainers/extracted/Graal_WaterGet/Graal WaterGet.exe
35dbd87ca7560712…	24576	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections	Trainers/extracted/AFKBOT/AFKBOT/AFK Bot.exe
fdab4020b62e1ef4…	24576	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections	Trainers/extracted/AFKSCROLLBOT/AFKSCROLLBOT/AFK Scroll Bot.exe
36e264ff1b9bd441…	40960	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections	Trainers/extracted/GTS/GTS.exe
812a0d32da379e75…	36864	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections	Trainers/extracted/GraalLoad/Graal Load.exe
5a4493d79d0619a9…	41317	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/extracted/autoprogs/AutoD.EXE
ebf4a5913132fa8f…	42041	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/extracted/autoprogs/AutoIdle.exe
7e427085717c47e2…	41072	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections	Trainers/extracted/autoprogs/AutoS.exe
2576cbeacd5ed30f…	130048	PE32 executable (GUI) Intel 80386, for MS Windows, 10 sections	Trainers/extracted/capslock/capslock.exe

Drive originals

Drive file ids omitted (private owner folder). Local paths + sizes + download status retained.

Path	Size	Status	SHA256
Trainers/AFKBOT.7z	5001	OK	f7ac393a1d6415c1…
Trainers/AFKSCROLLBOT.7z	5668	OK	bd46ee159f3ecaef…
Trainers/autoprogs.zip	111200	OK	0cfbfffde3b7ed08…
Trainers/capslock.zip	126297	OK	4c65b83d528c8d37…
Trainers/Graal_Auto_Key.zip	38024	OK	827d9b5549da0308…
Trainers/Graal_Digger.zip	39297	OK	27e051d29205b797…
Trainers/Graal_Miner.zip	39156	BLOCKED	—
Trainers/Graal_Prayer.zip	76631	OK	9c86172af8f52755…
Trainers/Graal_Random_Walker.zip	38442	OK	456991c2269be852…
Trainers/Graal_Water_Get.zip	38746	OK	8209e0ff526a55bd…
Trainers/Graal_WaterGet.zip	37327	OK	642e9849f5736ea9…
Trainers/GraalLoad.zip	18438	OK	ef05589392c3f77f…
Trainers/GTS.zip	8763	OK	2b0d01bade67865f…
Trainers/AutoPrograms by RpgWarrior/Graal_Auto_Key.zip	38024	OK	827d9b5549da0308…
Trainers/AutoPrograms by RpgWarrior/Graal_Digger.zip	39297	OK	27e051d29205b797…
Trainers/AutoPrograms by RpgWarrior/Graal_Miner.zip	39156	BLOCKED	—
Trainers/AutoPrograms by RpgWarrior/Graal_Prayer.zip	76631	OK	9c86172af8f52755…
Trainers/AutoPrograms by RpgWarrior/Graal_Random_Walker.zip	38442	OK	456991c2269be852…
Trainers/AutoPrograms by RpgWarrior/Graal_Water_Get_v1.0.zip	38746	OK	8209e0ff526a55bd…
Trainers/AutoPrograms by RpgWarrior/Graal_Water_Get_v1.1.zip	37327	OK	642e9849f5736ea9…

Extracted + notes inventory

Path	Size	MIME	Category	SHA256	file(1)
DEFENSE_PROPOSALS.md	16820	text/markdown	defense_notes	22cc5d5c07f47bef…	Unicode text, UTF-8 text
THREAT_TAXONOMY.md	16396	text/markdown	writeup	5a0b21e4e908d2e0…	Unicode text, UTF-8 text, with very long lines (320)

Path	Size	MIME	Category	SHA256	file(1)
Trainers/AutoPrograms by RpgWarrior/Graal_Miner.zip.BLOCKED.txt	371	text/plain	blocked_placeholder	b5ffa6b1d187bd35…	Unicode text, UTF-8 text
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Auto_Key/Graal Auto Key.exe	42793	application/vnd.microsoft.portable-executable	trainer_binary	463e8f757e82bc5a…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Digger/Graal Digger.exe	36111	application/vnd.microsoft.portable-executable	trainer_binary	876723732378b36e…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections ⚠️ Carved from corrupt zip; CRC fail; decompressed 36111 vs declared 41893 (truncated)
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Digger/Graal_Digger-Howto.txt	219	text/plain	writeup	2166abaa7cc1f196…	ASCII text, with CRLF line terminators
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Prayer/Graal Prayer.exe	43420	application/vnd.microsoft.portable-executable	trainer_binary	6959bcb8066efdee…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Prayer/data/data.exe	42676	application/vnd.microsoft.portable-executable	trainer_binary	c331eee0e985c903…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Random_Walker/Graal Random Walker.exe	43185	application/vnd.microsoft.portable-executable	trainer_binary	b41973d19fc0089c…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Water_Get_v1.0/Graal Water Get.exe	43483	application/vnd.microsoft.portable-executable	trainer_binary	4f95fa2af7d7db1c…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections

Path	Size	MIME	Category	SHA256	file(1)
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Water_Get_v1.1/Graal WaterGet.exe	41820	application/vnd.microsoft.portable-executable	trainer_binary	7482de4824d370f1…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/AutoPrograms by RpgWarrior/extracted/Graal_Water_Get_v1.1/Graal_WaterGet-Howto.txt	224	text/plain	writeup	7c11e4e42123e834…	ASCII text, with CRLF line terminators
Trainers/Graal_Miner.zip.BLOCKED.txt	274	text/plain	blocked_placeholder	f0372ca8e6db422b…	Unicode text, UTF-8 text
Trainers/extracted/AFKBOT/AFKBOT/AFK Bot Read Me.txt	1007	text/plain	writeup	2fbd034251d6fc81…	ASCII text, with CRLF line terminators
Trainers/extracted/AFKBOT/AFKBOT/AFK Bot.exe	24576	application/vnd.microsoft.portable-executable	trainer_binary	35dbd87ca7560712…	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections
Trainers/extracted/AFKSCROLLBOT/AFKSCROLLBOT/AFK Scroll Bot Read Me.txt	1320	text/plain	writeup	d2cbb74e05ab07b7…	ASCII text, with CRLF line terminators
Trainers/extracted/AFKSCROLLBOT/AFKSCROLLBOT/AFK Scroll Bot.exe	24576	application/vnd.microsoft.portable-executable	trainer_binary	fdab4020b62e1ef4…	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections
Trainers/extracted/GTS/GTS.exe	40960	application/vnd.microsoft.portable-executable	trainer_binary	36e264ff1b9bd441…	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections
Trainers/extracted/GraalLoad/Graal Load.exe	36864	application/vnd.microsoft.portable-executable	trainer_binary	812a0d32da379e75…	PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections
Trainers/extracted/GraalLoad/bomb.txt	1372	text/plain	graal_client_script	332cb8718c0f6098…	ASCII text, with CRLF line terminators
Trainers/extracted/Graal_Auto_Key/Graal Auto Key.exe	42793	application/vnd.microsoft.portable-executable	trainer_binary	463e8f757e82bc5a…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections

Path	Size	MIME	Category	SHA256	file(1)
Trainers/extracted/Graal_Digger/Graal Digger.exe	36111	application/vnd.microsoft.portable-executable	trainer_binary	876723732378b36e…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections ⚠️ Carved from corrupt zip; CRC fail; decompressed 36111 vs declared 41893 (truncated)
Trainers/extracted/Graal_Digger/Graal_Digger-Howto.txt	219	text/plain	writeup	2166abaa7cc1f196…	ASCII text, with CRLF line terminators
Trainers/extracted/Graal_Prayer/Graal Prayer.exe	43420	application/vnd.microsoft.portable-executable	trainer_binary	6959bcb8066efdee…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/Graal_Prayer/data/data.exe	42676	application/vnd.microsoft.portable-executable	trainer_binary	c331eee0e985c903…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/Graal_Random_Walker/Graal Random Walker.exe	43185	application/vnd.microsoft.portable-executable	trainer_binary	b41973d19fc0089c…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/Graal_WaterGet/Graal WaterGet.exe	41820	application/vnd.microsoft.portable-executable	trainer_binary	7482de4824d370f1…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/Graal_WaterGet/Graal_WaterGet-Howto.txt	224	text/plain	writeup	7c11e4e42123e834…	ASCII text, with CRLF line terminators
Trainers/extracted/Graal_Water_Get/Graal Water Get.exe	43483	application/vnd.microsoft.portable-executable	trainer_binary	4f95fa2af7d7db1c…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/autoprogs/AutoD.EXE	41317	application/vnd.microsoft.portable-executable	trainer_binary	5a4493d79d0619a9…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/autoprogs/AutoIdle.exe	42041	application/vnd.microsoft.portable-executable	trainer_binary	ebf4a5913132fa8f…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections

Path	Size	MIME	Category	SHA256	file(1)
Trainers/extracted/autoprogs/AutoS.exe	41072	application/vnd.microsoft.portable-executable	trainer_binary	7e427085717c47e2…	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
Trainers/extracted/capslock/capslock.exe	130048	application/vnd.microsoft.portable-executable	trainer_binary	2576cbeacd5ed30f…	PE32 executable (GUI) Intel 80386, for MS Windows, 10 sections
Trainers/extracted/capslock/readme.htm	7990	text/html	writeup	f055ad6e0a8c9838…	HTML document, ASCII text, with very long lines (815), with CRLF line terminators
defense-checklist.md	5153	text/markdown	defense_notes	fe5c170968bbec96…	Unicode text, UTF-8 text

Sibling static-extract artifacts

See `_static_extract/` (UPX unpacks, PE info, strings, AutoIt script dumps). Full hashes in `inventory.json`.

`recovered-hacks/remaining-2026-11-01.md`

Remaining trainer reads

Date: 2026-11-01. Static read of the four samples the behaviour note left open. No recovered PE was executed. A copy of the digger image was handed to UPX, which refused it. The caps-lock packer was not run. AutoD, Prayer, Graal Load, the water tools, and the walker are not re-opened here.

`[bin]` is a value read out of the image, the form, or the readme. `[inf]` is a reading of that evidence.

AFK Scroll Bot `[bin]`

Visual Basic 6, not packed. The form timer interval is 100 milliseconds. The slider is labelled Wait Time. Its saved block is the slider's own range (a ten, a five, and ones among the fields). No control is named for a shift, and the readme does not give one. The default phrase is `Scroll Text Here`.

The start procedure reads one control value and multiplies it by ten before storing a countdown. Each tick subtracts one. While the countdown is still at least one, the tick does not type. The only numeric control is that slider, and the readme says it is a number of seconds. Ten ticks of a 100 millisecond timer is one second. `[inf]`

Once the countdown has finished, every tick does this to the phrase:

- take the length minus one, from the end
- take one character from the start
- join those pieces with the long piece first
- write the result back over the phrase
- pass it to the runtime's send-keys call

The shift is one character: the first character moves to the end. It is in the timer procedure. It is not in the form and not in the readme. `scroll_phrase` in `reference_macros.py` replays that join. A one-character phrase stays as it is.

Immediately before send-keys, the constant 13 is passed to the runtime's text helper. That is the carriage return that submits the chat line. `[inf]` The status line is rebuilt as `Sent`, the tick count, and `AFK Messages`. That counter is not the shift.

Graal Text Sender `[bin]`

Visual Basic 6, not packed. "Graal Text Sender" v1.3.5 on the form. The form timer interval is 100 milliseconds. The resource directory holds an icon and a version block. The version block names the product and the file version `1.03.0005`. It does not name a key. There is no other resource.

The image declares `GetAsyncKeyState` and has a resolver stub for it. Nothing in the image calls that stub. The declare next to the name holds the library and a cache slot, not a virtual-key constant. The form strings are the mode titles, the timer prompts, and the about line. None of them names a key.

The key tested by the key-state check is not in the resources, and there is no call that would show it.

What the image does call is window lookup, and a send path that sets the window text and then posts key-down and key-up for Return. That submits a line. It is not the polled key.

capslock `[bin]`

The executable beside the readme. Ten sections, one of them the packer's. UPX does not recognize it. The code section is effectively random. The packer section is not.

The stub saves registers and jumps to a loop that adds a constant to a backward run of dwords. After that loop the bytes are not straight-line code. Relative calls that would leave the stub land on addresses that are not code. The stub does not call through the import table.

The import table lists the packer's loader trio plus `keybd_event`, `GetKeyboardType`, a registry query, and small COM and GDI helpers. `keybd_event` is not called from the stub. A version string says the program turns the caps-lock key on and off. The readme says the same, and that the command line accepts on or off. No socket wording appears.

The body stays packed. The stub was not executed to unpack it.

Digger `[bin]`

The zip was read again. The local headers are the howto, then the executable. The howto inflates completely: six numbered steps, equip a shovel, stand on grass, focus the client, click ok. No key and no wait. The executable's deflate stream ends after 36111 bytes. The local header declares 41893 uncompressed bytes. The central directory is a single entry, named for the executable, declaring 44013 uncompressed bytes and pointing at the start of the file. Both declarations are larger than what inflated. The inflated checksum matches neither header.

UPX reports an invalid overlay and will not unpack the partial image. The packed section is short of its declared size, and the resource section is entirely past the end of the file. The import directory is past the end as well.

The surviving bytes contain `AutoIt` and a broken v2.63 splash fragment, plus smashed pieces of the runtime's own words: a fragment of its key-name list, the name of its key-delay command, and the script-path token. Those are not a script. The script-start marker is absent. `Sleep` is absent. There is no chosen key and no wait count. They are not invented from the fragments.

Not in this note

The miner archive was not downloaded. It is the one the Drive abuse flag blocks.

`recovered-hacks/static-2026-11-01/autod.md`

AutoD

Static read only. The PE was not executed.

Sample	`AutoD.EXE` from the autoprogs archive
SHA-256 (packed)	`5a4493d79d0619a930413bd92bffd09935b11dcb33ce0e9e21270ab717d0be6d`
Runtime	UPX, then AutoIt v2.60. Ghidra 12.2 on the unpacked image: about 300 functions, no Winsock or HTTP imports
Script	Recovered statically from the unpacked image

What it does

It shows a yes/no box asking whether to run. No exits. It then tells the operator the program was made in 2001 and to use the tray icon to stop. The loop is `Sleep` 1 millisecond and `Send` of the letter `d`, forever.

The script has no registry, file, process, or network commands. The AutoIt runtime still imports `keybd_event`, `FindWindowA`, `SendMessageA`, registry calls, and `OpenProcess`. Those imports are the interpreter, not extra stages in this script. There is no `WriteProcessMemory` or `CreateRemoteThread`.

Defense

This is a dig-key macro, not a client patch. A server sees a stream of the dig action with almost no pause. Chat limits will not catch it. Gather cooldowns will.

`recovered-hacks/static-2026-11-01/autoidle.md`

AutoIdle

Static read only. The PE was not executed.

Sample	`AutoIdle.exe` from the autoprogs archive
SHA-256 (packed)	`ebf4a5913132fa8faca1647bb80aed2f57cde6f9e579bac71592093b3853a43b`
Runtime	UPX, then AutoIt v2.61. Ghidra 12.2 on the unpacked image: about 300 functions, no Winsock or HTTP imports
Script	Recovered statically from the unpacked image

What it does

One loop, then a goto back to the start. The body is only `Send` of `{LEFT}` and `{RIGHT}`. Most steps alternate one left and one right. A few stretches send the same arrow two or three times. There is no `Sleep`, no chat text, and no window check.

No registry, file, process, or network commands in the script. Same AutoIt runtime imports as AutoD (`keybd_event`, `FindWindowA`, registry, `OpenProcess`). No remote-thread or write-process imports.

Defense

This is an anti-idle movement jitter, not an AFK chat bot. Phrase lists and chat rate limits do not see it. A session that oscillates facing or position with no other actions does.

`recovered-hacks/static-2026-11-01/autos.md`

AutoS

Static read only. The PE was not executed.

Sample	`AutoS.exe` from the autoprogs archive
SHA-256 (packed)	`7e427085717c47e2e1f11e45abfc448613730cd1f20f41a8f6a29a8dfd3a88ad`
Runtime	UPX, then AutoIt v2.61. Ghidra 12.2 on the unpacked image: about 300 functions, no Winsock or HTTP imports
Script	Recovered statically. Three lines.

What it does

`Repeat` forever, `Send` of the two characters `s#`, `EndRepeat`. No sleep, no window check, no message box.

No registry, file, process, or network commands in the script. Runtime imports match the other autoprogs (`keybd_event` and friends). No injection imports.

Defense

A held attack or shortcut key, as fast as the AutoIt sender runs. Same class as AutoD: client keystrokes, server-side action cooldown.

`recovered-hacks/static-2026-11-01/capslock.md`

capslock

Static read only. The PE was not executed, and the packer was not run to unpack it.

Sample	`capslock.exe`
SHA-256	`2576cbeacd5ed30ff2fc02be1e8004d6fb4c380d1df3725fd7a09ab708b4ac7d`
Packing	Ten sections, including `.aspack`. UPX does not recognize it.
Ghidra	12.2 PUBLIC. Delphi language id on the stub. Four functions. Analysis did not recover the packed body.

What is visible without executing

Ghidra's import list for the stub is `LoadLibraryA`, `GetProcAddress`, `GetModuleHandleA`, `keybd_event`, `GetKeyboardType`, `RegQueryValueExA`, plus small COM and GDI helpers. There is no socket DLL, no `OpenProcess`, no `FindWindow`, no `WriteProcessMemory`.

A version-resource string in the same image says the program turns the caps-lock key on and off. The bundled readme (plain HTML, not a PE) says the same: CapsLock v1.00, a toggle, with command-line options. No Graal window title or game string appears in the packed image.

`LoadLibraryA` and `GetProcAddress` are the packer stub. They are not, by themselves, a second program. Nothing in the surviving strings names another payload.

Defense

Treat it as a caps-lock utility that was filed under trainers. It is not evidence of a Graal macro or a downloader. Do not execute it to finish the unpack. Game-server checks do not need a signature for it.

`recovered-hacks/static-2026-11-01/listserver.md`

Win32 ListServer

Static read only. The PE was not executed.

Sample	Packed `ListServer.exe` from the Server Lister archive
SHA-256 (packed)	`3e15bcc71ede779809bad4c05f46c6128793fed5135fec3120810d15a1928b40`
Image Ghidra loaded	UPX-unpacked copy of that PE
Ghidra	12.2 PUBLIC, Delphi, 3299 functions, headless analysis succeeded
Imports that matter	`WSAStartup`, `socket`, `inet_addr`, `RegEnumValueA`, `ShellExecuteA`

Directory behaviour

The image and the bundled `servers.ini` agree on a small local directory. The options section names a show-more URL and a list of server ids. Each server section has name, address, port, description, language, version, players, URL, and a favorite flag. The bundled example entry is loopback on lab port 14900. The file's own comment says a favorite entry is sent to the client with a `P` prefix so the client files it on the gold tab. Winsock listen/connect imports match a program that accepts client connections and answers with that list.

Credential hand-off

The same image carries a second cluster, next to `ShellExecuteA`:

- Registry path `Software\Microsoft\Windows\CurrentVersion\Internet Settings`, value name `Drag`, marker text `WindowsUser@`.
- Registry path `Software\Graal\Client`.
- Program name `iexplore`, verb `Open`.
- A base64 text beside those strings. Decoded, it is an `http` URL with a query string. The host is not recorded here.

`RegEnumValueA` is imported, which is what a walk of every value under the client key needs. Together this is the 2003 listserver that reads saved classic client logins and asks the local browser to open a third-party URL with that text attached. The marker under Internet Settings is how it avoids repeating the hand-off.

Do not run the packed or unpacked PE. Servertools should not shell out to it, ship it, or treat it as a listserver implementation. If it has already been started on an operator machine, saved classic client passwords on that machine should be rotated.

`recovered-hacks/static-2026-11-01/random-walker.md`

Graal Random Walker

Static read only. The PE was not executed.

Sample	`Graal Random Walker.exe`
SHA-256 (packed)	`b41973d19fc0089c0b5b59e7477ad008354a4e6132204692b89ad7b5d5ea8f48`
Runtime	UPX, then compiled AutoIt v2.64. The script sits in a compressed container in the overlay. Ghidra 12.2: about 300 functions. Imports include `keybd_event` and `FindWindowA`. No Winsock or HTTP.
Script	Recovered statically after the container was unpacked in memory only (the PE was not started).

What it does

It looks for a top-level window titled `Graal Auto`.

1. A message says to focus that window within a few seconds. It waits 2 seconds. If the window is not active it shows "use the key first" and exits.
2. It shows a "Graal Water Get" ok/cancel box (same prompt family as the water tool). Cancel exits.
3. It shows "click ok to start random walker", waits for `Graal Auto`, and activates it.
4. The loop sets the key delay to 200 milliseconds, draws a random integer from 0 through 5, and sends five of one key: 1 is Up, 2 is Down, 3 is Left, 4 is Right, 5 is `S`. Zero sends nothing and loops again. If the window is not active it shows a waiting box, sleeps 1 second, and resumes when the window is active.

No registry, file, or network commands in the script. `OpenProcess` and `VirtualAlloc` are present as AutoIt runtime imports. There is no `WriteProcessMemory` or `CreateRemoteThread`.

Defense

Movement and an occasional `S` burst, only while a window titled `Graal Auto` is in front. It is not a packet tool. Server-side speed and action limits still apply. A client whose window title is not `Graal Auto` never enters the loop.

`recovered-hacks/static-2026-11-01/README.md`

Static notes on original Graal tools and trainers

Date: 2026-11-01. Ghidra **12.2 PUBLIC** headless (`analyzeHeadless`) on Temurin **25.0.5+8**. Copies were unpacked with UPX where the packer was intact. No recovered PE was executed. No project database or binary is in this repo.

These notes live with the original-client research. They fill gaps the recovered tool and hack summaries do not answer: what the remaining AutoIt macros actually send, and how the Win32 listserver treats saved client logins. They are behaviour only.

Note	Sample	Why it was re-read
listserver.md	Win32 `ListServer.exe`	Prior summary describes a directory UI. It does not describe the registry read and browser hand-off.
autod.md	`AutoD.EXE`	Confirm the autoprogs family is a macro, and which key it sends.
autoidle.md	`AutoIdle.exe`	Same, for the idle program.
autos.md	`AutoS.exe`	Same, for the short program.
waterget.md	`Graal WaterGet.exe`	Remaining water-can loop: key and wait.
random-walker.md	`Graal Random Walker.exe`	Remaining walker: which keys, and whether it checks the client window.
capslock.md	`capslock.exe`	Packed loader in the trainers folder. Check for anything beyond a caps-lock toggle.

Not re-opened here

- `Graal_Miner` archives stay blocked by the Drive abuse flag. Not downloaded.
- Graal Digger remains the truncated UPX sample already summarized.
- Graal Prayer and `data.exe` script bodies stay encrypted in the prior summary. This pass did not open them again.
- Graal Load and AFK Scroll Bot were not imported in this pass. The earlier AFK Bot summary still covers the phrase bot; the scroll sibling and Graal Load stay on the prior string notes.

What a checker can use

- Dig, water, and attack macros are client keystrokes. A world checker cannot see them. Rate limits belong on the game server.
- AutoIdle never chats. It oscillates the arrow keys. Chat-phrase rules will not see it.
- The Win32 listserver is a desktop program. Do not run it. If it has been run on a machine that stored a classic client login, treat that login as exposed. The Seihai rebuild does not need a parser for it.

`recovered-hacks/static-2026-11-01/waterget.md`

Graal WaterGet

Static read only. The PE was not executed.

Sample	`Graal WaterGet.exe` (v1.1 archive)
SHA-256 (packed)	`7482de4824d370f1c9f5b73323470db90ae2fac69fca40e8ce14626a29481da0`
Runtime	UPX, then AutoIt v2.63 (script container marked v2.62). Ghidra 12.2: about 300 functions, no Winsock or HTTP imports
Script	Recovered statically. The bundled howto is the operator steps, not the key loop.

What it does

A message box asks the operator to click ok when ready, and names RpgWarrior. Cancel exits. It then sets the key delay to 300 milliseconds and loops: `Send {DOWN 2}`, `Send` the letter `a`, `Sleep` 2000 milliseconds.

The howto tells the operator to equip a water can, stand on a water source facing down, focus the client, and start the program. The script itself supplies the down tap and the use key. It does not open sockets or read the registry. The runtime still imports `keybd_event`, `FindWindowA`, registry functions, and `OpenProcess`, and it has a tray icon.

Defense

One use action every two seconds, plus a facing key. A per-account gather cooldown longer than that wait, or a yield cap, stops the farm. The client is unmodified.

`recovered-hacks/THREAT_TAXONOMY.md`

Threat taxonomy & attack-surface map — Drive `Hacks`

Generated: 2026-10-31 (static analysis only; no code execution)
Drive source: Owner private Google Drive folder named `Hacks` (URL and folder id omitted from this repo)
Local staging: `/workspace/recovered-hacks/`
Scope note: Google-drive MCP `list_folder` on the Hacks root currently returns **only** `Trainers/`. Parent Drive folder also holds `Tools`, `Graal clients`, `Graal playerworlds`, etc., but those are outside this Hacks tree. Classification below covers everything present in Hacks + local downloads as of this write.

Inventory status

Source	Result
Drive MCP early list	`Trainers/` + `Trainers/AutoPrograms by RpgWarrior/` (20 archive entries)
Local download	18/20 archives complete; 2 blocked by Google Safe Browsing
Sibling extract	Most archives already unpacked under `Trainers/extracted/` and `…/AutoPrograms by RpgWarrior/extracted/`
Static work product	`/workspace/recovered-hacks/_static_extract/` (PE notes, UPX-unpacked copies, string dumps) — **do not execute**

Archive checklist

Path	Size	Status	Contents (static)
`Trainers/AFKBOT.7z`	5001	OK	`AFK Bot.exe` + readme (Zorg)
`Trainers/AFKSCROLLBOT.7z`	5668	OK	`AFK Scroll Bot.exe` + readme (Zorg)
`Trainers/autoprogs.zip`	111200	OK	`AutoD.EXE`, `AutoIdle.exe`, `AutoS.exe` (UPX AutoIt)
`Trainers/capslock.zip`	126297	OK	`capslock.exe` + `readme.htm` (RJL Freeware)
`Trainers/Graal_Auto_Key.zip`	38024	OK	`Graal Auto Key.exe` (UPX AutoIt v2.64)

Path	Size	Status	Contents (static)
`Trainers/Graal_Digger.zip`	39297	OK	`Graal Digger.exe` (corrupt UPX overlay) + howto text in zip quirk
`Trainers/Graal_Miner.zip`	39156	**BLOCKED**	Drive 403 `cannotDownloadAbusiveFile` (malware/spam)
`Trainers/Graal_Prayer.zip`	76631	OK	`Graal Prayer.exe` + `data/data.exe` (both UPX AutoIt)
`Trainers/Graal_Random_Walker.zip`	38442	OK	`Graal Random Walker.exe`
`Trainers/Graal_Water_Get.zip` / `Graal_WaterGet.zip`	38746 / 37327	OK	Water-can macros (v1.0-style + older Howto)
`Trainers/GraalLoad.zip`	18438	OK	`Graal Load.exe` + `bomb.txt`
`Trainers/GTS.zip`	8763	OK	`GTS.exe` — Graal Text Sender (VB6)
`Trainers/AutoPrograms by RpgWarrior/*`	(dup set)	OK except Miner	Same RpgWarrior binaries; Miner also **BLOCKED**

Duplicates under `AutoPrograms by RpgWarrior/` match Trainers hashes for the shared tools (same SHA-256 for Auto Key, Prayer, Water Get, Random Walker, Digger, etc.).

Attack-class overview

Class	Items	Risk to Seihai / game servers	Present?
Game / server cheat — AFK chat bots	AFKBOT, AFKSCROLLBOT	Economy/online-time abuse; chat noise	Yes
Game / server cheat — input macros (AutoIt)	RpgWarrior suite, autoprogs	Resource farming; repetitive actions	Yes
Game / server cheat — chat / appearance script spam	GTS, GraalLoad + `bomb.txt`	Chat flood; cosmetic / social disruption	Yes
Suspected malware / unknown PE	Graal_Miner.zip (×2)	Host compromise **if** Safe Browsing correct	Yes (blocked)
Benign utility (misfiled)	capslock (RJL)	Negligible	Yes
Web exploit / auth bypass / priv-esc / network protocol / supply-chain / crypto / misconfig / RAT (confirmed)	—	—	**Not observed** in current Hacks tree

Group 1 — AFK chat bots (social / presence farming)

Items

- `Trainers/AFKBOT.7z → AFK Bot.exe` (Zorg, “Version 1.0”, ~2014 archive stamp; PE timestamp ~2002-era VB6)
- `Trainers/AFKSCROLLBOT.7z → AFK Scroll Bot.exe` (sequel; scroll-chat variant)

Likely attacker goals

- Keep a Graal client “active” / farm online-time on playerworlds without being at the keyboard.
- Inject rotating or scrolling chat phrases into the focused chat box.

Prerequisites

- Windows host with classic Graal client UI focused.
- User must click into Graal chat within a few seconds of start (documented for both tools).
- No server credential theft required; uses the already-logged-in client.

Blast radius

- **Playerworld / gserver:** policy violation, jail/ban risk; chat clutter; inflated online-time / AFK rewards if any.
- **Seihai client:** does not inject into Seihai; pattern still informs anti-AFK and chat rate design for servers Seihai connects to / servertools operates.
- **Host OS:** VB6 runtime (`MSVBVM60.DLL`) GUI apps; no network imports observed in static import dir (VB late-bound APIs may still call user32 SendKeys-style paths).

Static IoCs

Indicator	Detail
Filenames	`AFK Bot.exe`, `AFK Scroll Bot.exe`
UI strings	`AFK Bot created by Zorg`, `I am using an AFK Bot`, `I am AFK`, `Scroll Text Here`, `Sent 0 AFK Messages`
Warning string	`WARNING : AFK Bots are illegal on some playerworlds`
SHA-256	`35dbd87c…ff35` (AFK Bot), `fdab4020…2b84` (Scroll Bot)
PE	PE32 GUI, 3 sections, imports only `MSVBVM60.DLL`

Affected assets

- Classic Graal Windows client chat UI; playerworld online-time / anti-AFK rules.
- Indirect: any Seihai-compatible gserver that rewards presence or lacks chat rate limits (`seihai-servertools` ops).

Review mode

Doc-first (readmes are explicit). Light Ghidra only if validating whether SendKeys vs SendMessage is used — low priority vs Group 4/5.

Group 2 — RpgWarrior / Selzar AutoIt input macros (gameplay automation)

Items

Tool	Author cue	Intended action (from name/howto)
`Graal Auto Key.exe`	RpgWarrior set	Hold/repeat a key
`Graal Digger.exe`	Howto signed **Selzar**	Equip shovel → dig grass loop
`Graal Water Get.exe` / `Graal WaterGet.exe`	Howto **RpgWarrior**	Water-can at well, face down
`Graal Prayer.exe` + `data/data.exe`	dual-EXE package	Prayer / emote loop (name-based)
`Graal Random Walker.exe`	—	Random movement keys
`autoprogs`: `AutoD.EXE`, `AutoIdle.exe`, `AutoS.exe`	same AutoIt lineage	Dig / idle / (S)hortcut macros
`Graal_Miner.zip`	—	**Not downloaded** — see Group 4

Likely attacker goals

- Automate repetitive classic-Graal actions (dig, fill water, pray, walk, idle) for resource/AFK gains.
- Run unattended while the Graal window is focused (howto: start tool → focus Graal → confirm).

Prerequisites

- Windows + Graal client window; correct in-game item equipped / position (shovel, water can, well).
- Compiled **AutoIt v2.60–v2.64** runtimes packed with **UPX** (except Digger's UPX stream is corrupt).

Blast radius

- **Game economy / anti-bot:** farming bots; movement anomalies; identical key-timing patterns.
- **Client integrity:** no memory injection strings observed post-UPX unpack; primarily `keybd_event` / `mouse_event` / `SendMessageA` / `PostMessageA` / `OpenProcess` (AutoIt runtime capabilities).
- **Seihai:** macros target Win32 message/input APIs against the *legacy* client, not Seihai's Rust input path. Still a threat model input for server-side detection if operators run classic + Seihai populations together.

Static IoCs

Indicator	Detail
Packer	UPX (`UPX0`/`UPX1` sections); `$Info: … UPX …`
Runtime	the compiled-script banner for AutoIt 2.64, and the short version tokens for 2.60, 2.61 and 2.63
Vendor URL	the AutoIt vendor site (host omitted)
Marker	the AutoIt script-start marker, or the longer script-start banner (script body is not plaintext after the marker)
APIs (unpacked)	`keybd_event`, `mouse_event`, `SendMessageA`, `PostMessageA`, `FindWindowA`, `OpenProcess`, `RegOpenKeyExA`, `BlockInput`, `SETKEYDELAY`
Howto paths	`Graal_WaterGet-Howto.txt`, `Graal_Digger-Howto.txt`
Dual-bin	`Graal_Prayer/data/data.exe` (SHA-256 `c331eee0…6145`) alongside wrapper `6959bcb8…8860`
Shared PE TimeDateStamp	`1040239050` across several Mar 2003 RpgWarrior builds
Digger anomaly	Zip local vs central filename mismatch (`Graal_Digger-Howto.txt` vs `Graal Digger.exe`); `upx -d` → `OverlayException: invalid overlay size`

Affected assets

- Classic Graal client input surface; playerworld resource nodes (dig spots, wells).
- Detection hooks in gserver / servertools: repetitive action, impossible AFK activity, identical timing.

Review mode

Item	Mode
Howtos / names	**Doc-only**
Unpacked AutoIt family	**Ghidra optional** — recover encoded script; confirm no secondary payload
`Graal Prayer.exe` + `data/data.exe`	**Ghidra recommended** — dual binary unusual for this set
`Graal Digger.exe`	**Ghidra recommended** — corrupt UPX overlay / zip trick

Group 3 — Chat & appearance script spam (social / grief)

Items

- `Trainers/GTS.zip` → `GTS.exe` — **Graal Text Sender by DarkShadowz v1.3.5** (VB6)
- `Trainers/GraalLoad.zip` → `Graal Load.exe` (Hosler; the author's site, host omitted) + payload script `bomb.txt`

Likely attacker goals

- **GTS:** automated chat spam / word scroller / two-word repeat into Graal.
- **GraalLoad:** the player types a load command that names a script file; the tool feeds scripted lines (appearance commands + countdown taunt in `bomb.txt`).

Prerequisites

- Focused Graal chat; script files beside `Graal Load.exe`.
- `bomb.txt` uses classic client commands: `setnick`, `setbody`, `sethead`, `setcoat`, etc., then spam countdown to `!!BOOM!!`.

Blast radius

- Chat flood / harassment; temporary nick/body spam if server accepts those client commands.
- Low host-compromise signal from docs; GTS imports `OpenProcess`, `GetAsyncKeyState`, `SendMessageA` (via strings / late bind) — worth verifying no keylogging beyond hotkeys.

Static IoCs

Indicator	Detail
GTS strings	`Graal Text Sender`, `Word Scroller`, `Word Stay`, `Two Words Repeat`, `SendKeys1`
GTS SHA-256	`36e264ff…d0ab`
GraalLoad strings	a window title naming the tool and its author, a load command that names bomb.txt, and the author's site (host omitted)
GraalLoad SHA-256	`812a0d32…216e`
`bomb.txt` tokens	`setnick Bomb`, `You are going to die in`, `!!BOOM!!`, `setshield`, `setbody body5.png`

Affected assets

- Chat subsystem on playerworlds / listservers; Seihai chat UX parity (`documentation/chat-keys.md`) — servers should rate-limit / filter scripted floods.
- Appearance command surface if still honored server-side.

Review mode

- `bomb.txt`, GTS UI strings: **doc-only**.
- `GTS.exe`: **light Ghidra / VB decomp** — confirm `GetAsyncKeyState` / `OpenProcess` usage (hotkey vs keylog).
- `Graal Load.exe`: **doc + light RE** if load-protocol surprises appear.

Group 4 — Suspected malware (download blocked)

Items

- `Trainers/Graal_Miner.zip` (private Drive file — id omitted)
- `Trainers/AutoPrograms by RpgWarrior/Graal_Miner.zip` (private Drive file — id omitted)

Both: **403 `cannotDownloadAbusiveFile`** — *"identified as malware or spam"*; reported size **39156** bytes each (likely duplicates).

Likely attacker goals (inferred only)

- Name fits the RpgWarrior farming suite (mine resources), **or** Google's classifier saw a real trojan/dropper (AutoIt malware families are frequently abused).

Prerequisites / blast radius

- Unknown until a Safe Browsing override or alternate clean copy is obtained in an isolated lab.
- Treat as **host-compromise candidate** until proven otherwise — highest residual risk in this folder.

Static IoCs (metadata only)

Indicator	Detail
Placeholder	`Graal_Miner.zip.BLOCKED.txt` (both tree locations)
Size	39156 bytes (zip)
MIME	`application/zip`

Review mode

Ghidra / full malware RE required once bytes are available in an isolated environment. Do **not** run. Until then: metadata-only.

Group 5 — Benign utility (noise)

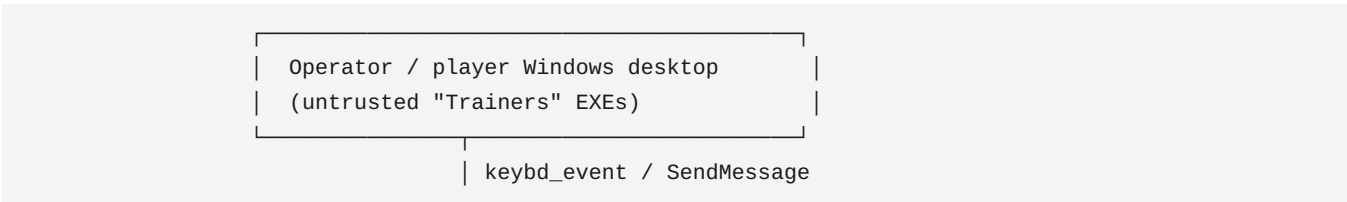
Item

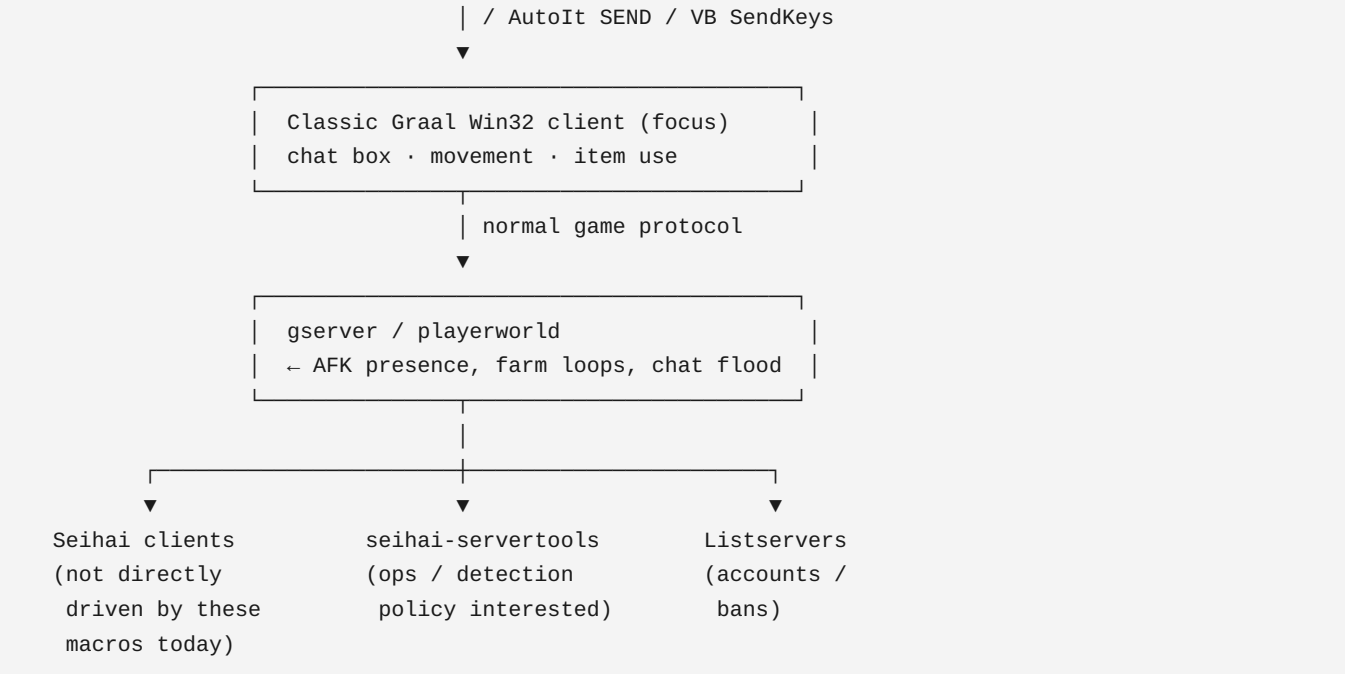
- `Trainers/capslock.zip` — RJL Software CapsLock v1.00 freeware (`readme.htm`, Aspack-packed PE).

Assessment

- Turns Caps Lock on/off; not a Graal cheat.
- IoCs: the utility vendor's site (host omitted), `keybd_event`, Aspack sections (`.aspack`).
- **Doc-only**; ignore for game threat model except "Trainers folder is a mixed junk drawer."

Attack-surface map (Seihai / servertools lens)





Not in this corpus: packet crafting against GServer, credential phishing kits, Seihai binary patching, supply-chain of Seihai crates, or web panel exploits.

RE triage matrix

Priority	Item	Why	Mode
P0	`Graal_Miner.zip` (both)	Google malware verdict; unknown payload	Full Ghidra/malware lab when obtainable
P1	`Graal Prayer.exe` + `data/data.exe`	Dual-EXE layout; hidden second stage risk	Ghidra after UPX unpack (already unpacked under `_static_extract/upx_unpacked/`)
P1	`Graal Digger.exe`	Corrupt UPX overlay + zip name mismatch	Ghidra / manual unpack
P2	`GTS.exe`	`OpenProcess` + `GetAsyncKeyState` strings	Light Ghidra / VB decomp
P2	`autoprogs` AutoD / AutoIdle / AutoS	Encoded AutoIt scripts; confirm payload = macros only	Ghidra or AutoIt script recovery
P3	Remaining RpgWarrior AutoIt single EXEs	Same family as P2	Batch script recovery; skip deep RE if scripts clean
P3	`Graal Load.exe`	Simple script typer; known `bomb.txt`	Doc + optional light RE
—	AFK Bot readmes, howtos, `bomb.txt`, capslock readme	Fully explained in text	**Doc-only**

Top risks (summary ranking)

1. **Graal_Miner.zip** — Safe Browsing malware block; unreviewed binary.
2. **Graal Prayer dual EXE** — second PE in `data/` elevates packing/dropper suspicion.
3. **Graal Digger** — intentional or accidental corrupt packer stream; treat carefully.
4. **GTS (Graal Text Sender)** — process/open + key-state APIs; chat weaponization.
5. **AFK Bot / Scroll Bot + GraalLoad/bomb** — high operational impact on playerworlds (presence fraud + harassment) even though host risk looks lower.

Method notes

- Prefer local `/workspace/recovered-hacks` once populated; Drive MCP used for early listing and blocked-file confirmation.
- Analysis: `file`, `sha256sum`, PE header parse, `strings`, `upx -d` on **copies**, archive listing (`unzip -l` / `7z l`), readme/howto text.
- **No execution** of recovered PE/AutoIt/VB binaries.
- Re-run this taxonomy if the Hacks Drive root gains folders beyond `Trainers/`.

RE addendum (2026-10-31)

Headless Ghidra + static PE work on P1/P2 trainers is filed under:

- **Synthesis (IOCs / behavioral tells):** `TRAINER_RE_NOTES.md`
- **Per-binary defense summaries:** `ghidra-analysis/` — `Graal_Prayer/`, `Graal_Prayer_data/`, `Graal_Digger/`, `GTS/`, `AFK_Bot/`

Confirmed statically: Prayer/`data.exe` are twin AutoIt v2.64 UPX macros (no network / no classic injection imports; encrypted script overlays differ). GTS is VB6 chat spam with string-visible `OpenProcess` + `GetAsyncKeyState` + `FindWindow*`. AFK Bot is string-rich VB6 presence spam. **Digger remains blocked** by truncated zip/UPX (hash still usable; imports not recoverable). Miner still skipped.

`recovered-hacks/TRAINER_RE_NOTES.md`

Trainer reverse-engineering notes (defense)

Date: 2026-10-31
Scope: Static analysis of highest-risk Graal trainer PEs under `/workspace/recovered-hacks/`
Method: `file` / SHA-256 / PE parse / `strings` / `objdump` / `upx -t` on copies / Ghidra 12.2 headless (`tools/ghidra/analyze_tool.sh`)
Constraint: No execution of trainers/bots. `Graal_Miner.zip` skipped (Safe Browsing block).

Per-binary writeups: `/workspace/recovered-hacks/ghidra-analysis/*/SUMMARY.md`

Binaries analyzed

Target	Path (primary)	Ghidra out dir	Result
Graal Prayer.exe	`Trainers/extracted/Graal_Prayer/Graal Prayer.exe` (analyzed UPX-unpacked twin)	`ghidra-analysis/Graal_Prayer/`	AutoIt v2.64 input macro runtime
data/data.exe	`Trainers/extracted/Graal_Prayer/data/data.exe` (unpacked twin)	`ghidra-analysis/Graal_Prayer_data/`	Second AutoIt v2.64 script PE
Graal Digger.exe	`Trainers/extracted/Graal_Digger/Graal Digger.exe`	`ghidra-analysis/Graal_Digger/`	Truncated UPX; AutoIt v2.63 cues only
GTS.exe	`Trainers/extracted/GTS/GTS.exe`	`ghidra-analysis/GTS/`	VB6 chat spam; FindWindow/OpenProcess/GetAsyncKeyState
AFK Bot.exe	`Trainers/extracted/AFKBOT/AFKBOT/AFK Bot.exe`	`ghidra-analysis/AFK_Bot/`	VB6 AFK chat bot (Zorg)

Cross-cutting defense IOCs

Hashes (SHA-256)

Sample	SHA-256
Graal Prayer.exe (packed)	`6959bcb8066efdee0dd842b6ee9d769cf93230d90a4ebca04786bfddb7f88860`
data.exe (packed)	`c331eee0e985c903db933432a62a4972b835aefe9a4832d9ce5705c694a86145`
Graal Prayer.exe (UPX-unpacked)	`a348ecafc3fdf3069dcaf9dd6d776c3f83c6f6d4f547ccf09d94f3694d952074`
data.exe (UPX-unpacked)	`b72e36502b2638ceeb9642ecb8d9da3c08c880175e60ec782a48b9ae93172a4c`
Graal Digger.exe (truncated carve)	`876723732378b36e8fd31d09120ec5a1b584cf46fb672ab9d5cf7a0a4f8b56d4`
GTS.exe	`36e264ff1b9bd441b278ec4afc265c7991dc2f9ae065f721d178f59c74d2d0ab`
AFK Bot.exe	`35dbd87ca7560712aae0e1bb3c4b4861e5233eac32299170af25eaa71c13ff35`

Filenames / paths

- `Graal Prayer.exe`, `data\data.exe` (paired)
- `Graal Digger.exe`, `Graal_Digger-Howto.txt`
- `GTS.exe` / UI title **Graal Text Sender**
- `AFK Bot.exe` (and sibling `AFK Scroll Bot.exe`)

Packer / runtime signatures

Family	Markers
RpgWarrior AutoIt macros	UPX `UPX0`/`UPX1`; the compiled-script banner for AutoIt 2.64; the AutoIt script-start marker; the AutoIt vendor site (host omitted); PE TimeDateStamp `1040239050` on Prayer pair
Digger (damaged)	UPX + AutoIt **v2.63** splash fragment; PE TimeDateStamp `1012939264`
GTS / AFK	`MSVBVM60.DLL`; VB6 `.vbp` absolute build paths under `...\VB98\...`

API / behavioral signatures (host)

Tool	Notable APIs / tells
Prayer / data.exe	`keybd_event`, `mouse_event`, `SendMessageA`, `PostMessageA`, `FindWindowA`, `SetTimer`, `BlockInput`, `OpenProcess`, `CreateProcessA`, `VirtualAlloc` — **no** `WriteProcessMemory` / `CreateRemoteThread` / Winsock
GTS	Late-bound `FindWindowA`/`Ex`, `OpenProcess`, `GetAsyncKeyState`, `SendMessageA`, `SendKeys1`, multiple **Set Timer** spam modes; window title token `Graal`
AFK Bot	VB6 + `Timer1`; default phrases `I am AFK` / `I am using an AFK Bot`; warning string about illegality on playerworlds
Digger	Intact behavior not recoverable; howto implies shovel dig loop while Graal focused

Network

No `WSAStartup` / WinINet / WinHTTP imports observed on Prayer, data.exe, GTS, or AFK Bot static surfaces. Risk class is **client-side input/chat abuse**, not standalone C2 from these PEs.

Process injection

AutoIt runtimes import `OpenProcess` + `VirtualAlloc` (capability noise). Classic injection APIs (`WriteProcessMemory`, `CreateRemoteThread`, hooks) **absent** from Prayer/data import tables. GTS `OpenProcess` pairs with window PID lookup — consistent with SendMessage automation, not proven shellcode inject.

Findings by priority target

1. Graal Prayer + data/data.exe

- Dual-EXE package is **two compiled AutoIt scripts**, not a demonstrated PE dropper (no path strings linking them).
- Encrypted overlays differ (~924 B vs ~180 B) after a shared RpgWarrior overlay header — treat both hashes as IOCs.
- Script plaintext not recovered (AutoIt v2 encrypted overlay); defense value is API/string/hash cluster.

2. Graal Digger.exe

- Zip yields truncated deflate of the PE (36111 / 41893 bytes). `upx` refuses unpack; Ghidra sees only UPX stub.
- Still classifiable as AutoIt+UPX dig macro via strings + Selzar howto.
- **Blocker:** missing ~5.7 KiB prevents import recovery and script recovery.

3. GTS.exe

- Confirmed DarkShadowz **v1.3.5** Graal Text Sender.
- Strongest combo IOC: VB6 + `OpenProcess` + `GetAsyncKeyState` + `FindWindow*` + product strings and the author's homepage (host omitted).
- Timer UI frames map directly to chat flood modes (scroller, to-all, shout).

4. AFK Bot.exe

- Representative AFK presence bot; string-rich, import-poor.
- Server/chat defenses: match default AFK phrases and periodic identical sends; host: filename + Zorg UI strings.

Detection ideas (defensive only)

1. **Hash allow/deny** for the table above (packed + unpacked AutoIt where endpoints unpack UPX).
2. **YARA-ish concepts:** UPX section names plus the compiled AutoIt 2.6 banner and the AutoIt script-start marker; or VB6 + `Graal Text Sender` / `GetAsyncKeyState` + `OpenProcess`.
3. **Window title:** `Graal Text Sender`, `AFK Bot`, AutoIt tray/debug window class behaviors.
4. **Server-side:** rate-limit shout/to-all; flag periodic identical AFK lines; dig/prayer action frequency caps on classic worlds.
5. **Do not** rely on Digger imports until a complete PE is obtained.

Blockers

Item	Blocker
Graal_Miner.zip	Safe Browsing — not downloaded/executed (per policy)
Graal Digger.exe	Truncated zip payload; corrupt UPX overlay; Ghidra import table empty
AutoIt script bodies (Prayer/data)	Encrypted overlays; no plaintext script recovered in this pass
GTS/AFK named Win32 imports in PE IDT	Hidden behind VB runtime / `DllFunctionCall` — use strings

Artifact index

```
/workspace/recovered-hacks/ghidra-analysis/Graal_Prayer/SUMMARY.md
/workspace/recovered-hacks/ghidra-analysis/Graal_Prayer_data/SUMMARY.md
/workspace/recovered-hacks/ghidra-analysis/Graal_Digger/SUMMARY.md
/workspace/recovered-hacks/ghidra-analysis/GTS/SUMMARY.md
/workspace/recovered-hacks/ghidra-analysis/AFK_Bot/SUMMARY.md
/workspace/recovered-hacks/TRAINER_RE_NOTES.md          (this file)
/workspace/recovered-hacks/_static_extract/upx_unpacked/  (Prayer/data unpacked inputs)
```